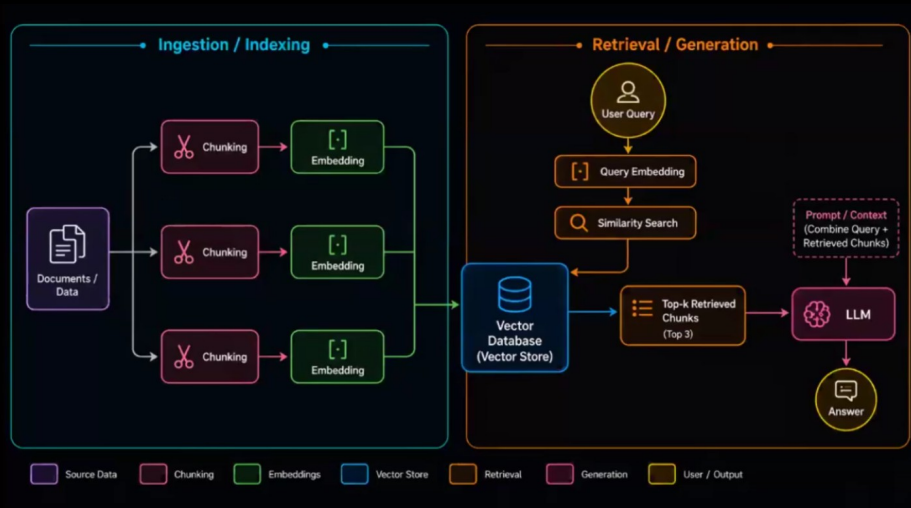


RAG INTRODUCTION



- ① Data Ingestion
- ② Retrival
- ③ Generation

RAG is a technique that helps an LLM answer questions using external data such as PDFs, documents, websites, databases, or company knowledge bases.

What is RAG?

RAG stands for Retrieval-Augmented Generation.

Retrieval = finding relevant information from documents.

Augmentation = adding that retrieved information as extra context to the user question

Generation = LLM generates the final answer using that context

Example

User asks:
"What is the refund policy of our company"

The LLM may not know this because company policy is private data.

RAG does this:

- 1. It searches the company policy document.
- 2. It retrieves the relevant refund policy section.
- 3. It adds that retrieved section as extra context to the user question.
- 4. It gives the context + question to LLM
- 5. The LLM generates the final answer based on that document.

Why RAG?

We use RAG because normal LLMs have some limitations.

Problem 1: LLMs do not have private data

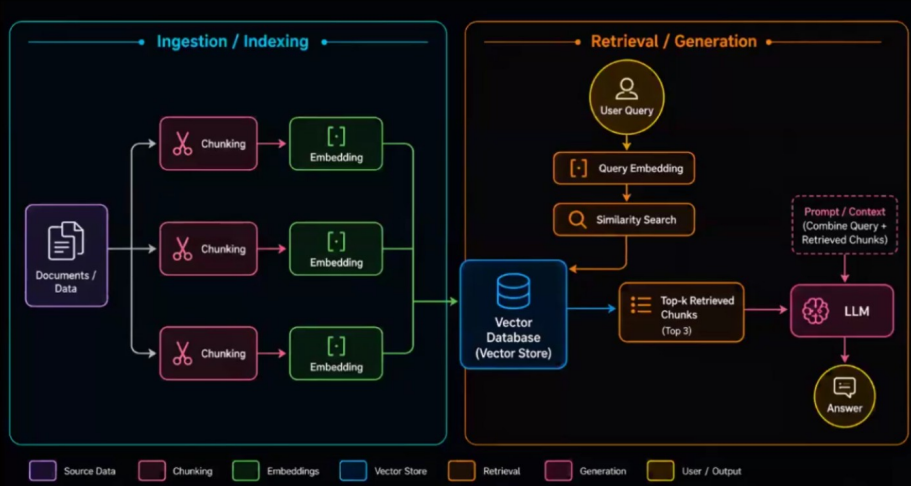
Problem 2: LLMs may not have latest data

Problem 3: LLM can hallucinate
Sometimes LLMs generate confident but wrong answers.
RAG reduces hallucination because the answer is based on retrieved context.

Problem 4: Fine-tuning is not always the right solution
For every new document, we cannot fine-tune the model again and again.
RAG is better when data changes frequently

Pipeline to Implement RAG

RAG helps an LLM answer questions using external documents. First, we parse the documents and extract clean text. Then we split the text into chunks, convert chunks into embeddings, and store them in a vector database. When the user asks a question, we retrieve the most relevant chunks and give them to the LLM along with the question.



RAG vs Fine-tuning

Aspect	RAG	Fine-tuning
How it works	Retrieves external info before generating	Learns from new data during training
Data requirement	Needs a good knowledge base or index	Needs a high-quality, labeled dataset
Flexibility	Great for dynamic, changing content	Great for fixed, well-defined tasks
Setup cost	Lower training cost; more infra setup	Higher training cost upfront
Performance	High recall, but may rely on retrieval quality	High precision, if fine-tuned well
Latency	Slightly slower due to retrieval step	Faster responses after deployment